

Fundamentos de Administração de Servidores

Guia Teórico, Procedimentos de Laboratório e Missão Avaliativa Prática

Um servidor corporativo é composto pelo conjunto indivisível entre hardware de alta confiabilidade e serviços computacionais (daemons) projetados para fornecer recursos a clientes remotos via rede. O papel do administrador de servidores (SysAdmin) consiste em assegurar disponibilidade ininterrupta, integridade operacional e segurança estrita contra acessos não autorizados.

1. Tipos de Servidores: Servidores Web Corporativos

MÓDULO 1.1

Servidores web processam requisições sob a pilha de protocolos HTTP e HTTPS (portas padrão TCP 80 e 443). No ecossistema moderno, atuam tanto na entrega de ativos estáticos quanto na função de **Proxy Reverso** e **Terminador TLS**, protegendo aplicações internas contra sobrecarga direta.

```
[Cliente Web] --(HTTPS / Porta 443)--> [Servidor Web (Nginx)] --(Proxy HTTP / Porta 3000)--> [App Backend Node/Python]
```

1.1. Comparativo de Motores: Nginx vs. Apache HTTP Server

Característica	Nginx (Engine-X)	Apache HTTP Server (httpd)
Modelo de Concorrência	Assíncrono, orientado a eventos (<i>Event-driven</i>). Uma única thread atende milhares de conexões.	Baseado em processos/threads concorrentes (MPM Prefork, Worker ou Event).
Consumo de Memória	Extremamente estável e previsível, mesmo sob cargas de dezenas de milhares de clientes.	Pode escalar de forma agressiva sob concorrência devido à alocação de threads por conexão.
Configuração Descentralizada	Não suporta arquivos por diretório. Toda a configuração é centralizada nos blocos de <code>/etc/nginx/</code> .	Suporta arquivos <code>.htaccess</code> por diretório, permitindo autonomia sem reinicializar o daemon.
Casos de Uso Primários	Edge Proxy, Balanceador de Carga, Cache de Alta Performance e Microsserviços.	Aplicações monolíticas clássicas (LAMP com <code>mod_php</code>) e hospedagens compartilhadas.

Ponto Crítico em Produção: Ciclo de Vida do HTTP

Ao receber uma solicitação, o servidor analisa cabeçalhos (Host, User-Agent, Cookies), resolve o arquivo local ou encaminha via socket (FastCGI, WSGI, `proxy_pass`) e responde com códigos de status padronizados: **2xx** (Sucesso), **3xx** (Redirecionamento), **4xx** (Erros do Cliente como 404 Not Found ou 403 Forbidden) e **5xx** (Falhas do Servidor/Aplicação como 500 ou 502 Bad Gateway).

1.2. Servidores de Banco de Dados: Transacionalidade e Persistência

Diferente dos servidores web que priorizam conexões rápidas e sem estado (*stateless*), os servidores de banco de dados gerenciam estado persistente sob garantias estritas de integridade transacional (**ACID**: Atomicidade, Consistência, Isolamento e Durabilidade).

SGBDs Relacionais (RDBMS)

Exemplos: PostgreSQL (porta TCP 5432), MySQL / MariaDB (porta TCP 3306).

Foco: Esquemas rígidos com chaves estrangeiras, consultas relacionais complexas em SQL e integridade corporativa financeira e transacional.

SGBDs Não Relacionais (NoSQL)

Exemplos: MongoDB (TCP 27017), Redis (TCP 6379), Cassandra.

Foco: Esquemas flexíveis em documentos (JSON/BSON), chave-valor em memória RAM para cache ultra veloz ou particionamento horizontal maciço.

1.3. Servidores de Arquivos: Compartilhamento em Rede Corporativa

Servidores de arquivos centralizam o armazenamento de documentos corporativos, backups e volumes de dados, oferecendo cotas por usuário e controle concorrente de bloqueio de escrita (*file locking*).

Protocolo / Serviço	Porta Padrão	Ecossistema Nativo	Propósito Principal
SMB / CIFS (Samba)	TCP 445	Microsoft Windows / Linux	Compartilhamento multiplataforma de pastas e impressoras; integração com domínios de rede.
NFS (Network File System)	TCP 2049	UNIX / Linux nativo	Montagem transparente de diretórios remotos com alta performance entre instâncias Linux.
SFTP / SCP (SSH)	TCP 22	Universal Seguro	Transferência criptografada de arquivos de sistema e despejos administrativos de rotina.

Segurança de Rede em Servidores de Armazenamento e Dados

Nunca exponha portas de bancos de dados (ex: 3306, 5432) ou compartilhamento de arquivos (445, 2049) diretamente para a internet pública. Estes serviços devem operar estritamente isolados em sub-redes privadas (VLANs), protegidos por firewalls e acessíveis exclusivamente via VPN administrativa ou túneis SSH autenticados por chaves assimétricas.

A decisão do sistema operacional condiciona o custo total de propriedade (TCO), os modelos de automação de tarefas e a governança de credenciais da empresa.

2.1. Análise Comparativa Estrutural

Vetor de Comparação	Distribuições Linux (Ubuntu Server, Debian, RHEL)	Microsoft Windows Server (2022 / 2025)
Modo Operacional	Quase exclusivamente <i>Headless</i> (apenas linha de comando SSH). Consumo de RAM ocioso < 500 MB.	Suporta Server Core (CLI) ou Desktop Experience (GUI completa). Consumo inicial > 2 GB RAM.
Modelo de Configuração	Baseado em arquivos de texto plano legíveis sob <code>/etc/</code> . Controle de versão via Git e automação.	Configuração baseada no Registro do Windows (Registry), consoles MMC e políticas de grupo (GPO).
Gerenciamento de Identidade	Autenticação local (<code>/etc/passwd</code>) ou centralizada via OpenLDAP, FreeIPA, SSSD e Kerberos.	<i>Active Directory Domain Services</i> (AD DS) nativo, com estrutura em florestas, árvores e OUs.
Linguagem de Shell & Script	GNU Bash, Zsh, AWK, sed e ferramentas core POSIX. Foco em pipelines de texto Unix.	PowerShell orientado a objetos (.NET). Transmite objetos tipados entre os cmdlets.
Modelo de Licenciamento	Open Source (GPL/BSD). Suporte corporativo opcional por assinatura (ex: Red Hat, Canonical).	Licenciamento proprietário calculado por núcleos de processador (Cores) + licenças de acesso (CALs).

2.2. Por que o Linux Domina a Nuvem e Infraestruturas Modernas?

1. Densidade e Containerização A arquitetura do kernel Linux provê <i>namespaces</i> e <i>cgroups</i> nativos, permitindo que contêineres Docker rodem sem sobrecarga de virtualização adicional.	2. Infraestrutura como Código (IaC) Por armazenar todas as definições em arquivos de texto plano, servidores Linux integram-se com ferramentas de orquestração como Ansible, Puppet e Terraform.
Visão Holística do Administrador Ambientes corporativos maduros raramente são monotemáticos. O padrão da indústria adota ambientes híbridos : Windows Server gerenciando autenticação e estações desktop corporativas via Active Directory, enquanto servidores Linux sustentam bancos de dados, clusters Kubernetes e servidores web.	

3.1. Arquitetura de Contas e Base de Dados Local

No Linux, a governança de identidades locais apoia-se em três arquivos essenciais sob o diretório raiz de configuração `/etc`:

/etc/passwd Registra metadados públicos: UID, GID primário, comentário, diretório <i>home</i> e interpretador de comandos (shell padrão) do usuário.	/etc/shadow Acessível apenas pelo <code>root</code> . Guarda os hashes criptográficos das senhas (SHA-512) e parâmetros de expiração de credenciais.	/etc/group Mapeia os grupos de segurança, seus respectivos identificadores (GIDs) e os membros adicionais pertencentes a cada agrupamento.
--	--	--

3.2. Decomposição de Linha: Entendendo o `/etc/passwd`

```
analista_maria:x:1001:1002:Maria Engenharia:/home/analista_maria:/bin/bash
[1]      [2] [3] [4]      [5]      [6]      [7]
```

- **[1] Username:** Nome da conta utilizado no processo de autenticação.
- **[2] Flag de Senha:** O caractere `x` indica que a senha real reside protegida no `/etc/shadow`.
- **[3] UID:** Identificador Numérico de Usuário (0 para `root`; 1 a 999 para `daemons`; 1000+ para humanos).
- **[4] GID Primário:** Identificador do grupo principal associado à conta.
- **[5] GECOS:** Campo de metadados descritivos (nome completo, departamento, ramal telefônico).
- **[6] Home Directory:** Ponto de montagem da pasta pessoal onde residem seus arquivos privados.
- **[7] Shell Padrão:** Binário executado no `login` (ex: `/bin/bash` ou `/usr/sbin/nologin` para serviços).

3.3. O Paradigma do Superusuário e Elevação Segura (`sudo`)

Trabalhar conectado diretamente como superusuário `root` viola princípios elementares de segurança, pois elimina a trilha de auditoria e eleva o risco de comandos destrutivos acidentais. A ferramenta `sudo` (Superuser Do) resolve este problema:

Prática	Login Direto como Root	Elevação Pontual via Sudo
Rastreabilidade	Nula. O log apenas registra que o usuário 'root' executou a ação.	Total. O log <code>/var/log/auth.log</code> registra qual usuário real elevou privilégios e qual comando digitou.
Compartilhamento	Exige que múltiplos administradores conheçam a mesma senha mestra.	Cada administrador usa sua própria senha pessoal. A revogação é feita removendo-o do grupo <code>sudo</code> .

3.4. Anatomia da Máscara de Permissões

Ao listar um arquivo ou pasta com `ls -l`, o sistema exibe uma sequência de 10 caracteres representando a tipologia e os acessos das 3 classes de segurança:

```
- | r w x | r - x | - - - ==> Arquivo com permissão 750
Tipo | Dono (U) | Grupo (G) | Outros (O)
```

3.5. A Matemática dos Pesos Binários e Notação Octal

Cada caractere representa um bit dentro de um registrador binário de 3 bits para cada entidade:

Permissão	Bit	Valor Decimal	Semântica em Arquivos	Semântica em Diretórios (Pastas)
Leitura (r)	100 ₂	4	Permite visualizar o conteúdo (ex: cat, less).	Permite listar os nomes dos arquivos contidos (ls).
Escrita (w)	010 ₂	2	Permite modificar ou truncar o conteúdo interno.	Permite criar, renomear ou deletar arquivos dentro da pasta.
Execução (x)	001 ₂	1	Permite rodar o arquivo como programa ou script.	Permite adentrar na pasta (cd) e consultar metadados.

3.6. Casos Clássicos de Aplicação de Máscaras Octais

- `chmod 700 /root/.ssh` $\rightarrow$ `$ rwx-----`: Acesso exclusivo ao proprietário. Mandatório para chaves privadas.
- `chmod 644 /var/www/index.html` $\rightarrow$ `$ rw-r--r--`: Dono lê e escreve; todos os demais apenas leem.
- `chmod 750 /srv/projetos` $\rightarrow$ `$ rwxr-x---`: Dono tem controle total; equipe lê/entra; terceiros barrados.
- `chmod 770 /dados_compartilhados` $\rightarrow$ `$ rwxrwx---`: Espaço de trabalho colaborativo restrito à equipe.

Regra de Ouro da Segurança Operacional: Jamais use `chmod 777`

Atribuir permissão 777 concede controle irrestrito de modificação e execução a qualquer processo ou usuário não privilegiado no sistema. É a principal causa de contaminação cruzada e invasões em servidores compartilhados.

Monitorar um servidor envolve coletar métricas contínuas para detectar gargalos operacionais antes que eles provoquem indisponibilidade para os usuários.

4.1. As Quatro Variáveis Essenciais de Desempenho

1. Carga de Processador (CPU) Métrica-chave: <i>Load Average</i> (médias de 1, 5 e 15 minutos obtidas via <code>uptime</code>). Conceito: Representa a quantidade média de processos que estão em execução ou aguardando disponibilidade da CPU. Em um servidor de 4 núcleos (cores), um <i>Load Average</i> acima de 4.00 indica enfileiramento e saturação.	2. Gestão de Memória RAM Métrica-chave: Coluna <code>available</code> do utilitário <code>free -h</code>. Conceito: O Linux utiliza memória ociosa como cache de disco (<i>buff/cache</i>). Memória em cache não é memória perdida: o kernel a libera instantaneamente caso processos de aplicação precisem de espaço.
3. Armazenamento e E/S (Disk I/O) Métrica-chave: Ocupação percentual via <code>df -h</code> e taxa de inodes com <code>df -i</code>. Conceito: Discos cheios travam bancos de dados e serviços web imediatamente. Além do espaço em gigabytes, o esgotamento da tabela de metadados (inodes) impede a criação de novos arquivos mesmo com espaço livre.	4. Interfaces e Tráfego de Rede Métrica-chave: Monitoramento de portas abertas (<code>ss -tulpn</code>) e pacotes descartados. Conceito: Avalia conexões concorrentes ativas e saturação da largura de banda da placa de rede física ou virtual.

4.2. Principais Comandos de Diagnóstico Imediato

Comando	Parâmetros Recomendados	Diagnóstico Fornecido
<code>free</code>	<code>free -h</code>	Memória física total, alocada, buffers/cache e volume em área de troca (Swap).
<code>df</code>	<code>df -hT /</code>	Espaço ocupado e livre nas partições em formato legível (GB/MB) e tipo de filesystem (ext4, xfs).
<code>top / htop</code>	<code>htop</code> ou <code>top -b -n 1</code>	Painel dinâmico dos processos em tempo real ordenáveis por uso de memória ou processador.
<code>ss</code>	<code>ss -tulpn</code>	Lista sockets TCP e UDP em modo escuta (listening), com portas e PIDs dos serviços donos.

4.3. O Subsistema de Logs no Linux Moderno

No Linux estruturado sob o systemd, o componente responsável por interceptar todas as mensagens emitidas pelo kernel, drivers e daemons é o systemd-journald. Ele armazena dados binários indexados que podem ser consultados via journalctl, enquanto serviços legados e servidores de aplicação mantêm arquivos de texto plano em /var/log/.

4.4. Principais Arquivos de Log e seus Papéis

Arquivo de Log	Conteúdo e Importância Forense
/var/log/auth.log (ou secure)	Registra todas as tentativas de autenticação: logons via SSH bem-sucedidos ou rejeitados, invocações do sudo e criação de contas.
/var/log/nginx/access.log	Registra cada conexão HTTP recebida pelo servidor web, contendo IP de origem, data/hora, método, URI requisitada, código HTTP e User-Agent.
/var/log/nginx/error.log	Grava anomalias críticas do servidor web: arquivos não encontrados (404), bloqueios de permissão (403) e falhas de upstream.
/var/log/syslog (ou messages)	Arquivo geral de registros operacionais do sistema operacional e serviços sem logs dedicados.

4.5. Operações de Inspeção em Tempo Real

O SysAdmin analisa logs com ferramentas de fluxo para identificar ataques de força bruta ou falhas em serviços no exato momento em que ocorrem:

```
# Seguir registros em tempo real enquanto requisições acontecem (-f)
tail -f /var/log/nginx/access.log

# Filtrar apenas erros ou acessos que resultaram no código HTTP 404
grep " 404 " /var/log/nginx/access.log

# Inspeccionar os últimos 30 eventos da unidade do serviço Nginx via journalctl
journalctl -u nginx -n 30 --no-pager
```

Boas Práticas: Rotação de Logs (logrotate)

Sem tratamento, arquivos de log crescem continuamente até esgotar a partição de disco. O daemon logrotate compacta e arquiva periodicamente logs antigos (ex: access.log.1.gz), descartando versões com mais de \$N\$ semanas conforme configurado em /etc/logrotate.d/.

O instrutor demonstrará no terminal do projetor os comandos fundamentais de provisionamento de usuários, isolamento de pastas de projetos, ativação de serviços web e inspeção de métricas de infraestrutura.

Parte 1: Governança de Grupos e Isolamento

```
# 1. Criação do grupo departamental 'devs'
sudo groupadd devs

# 2. Criação da conta 'devjunior' associada ao grupo e senha inicial
sudo useradd -m -s /bin/bash -g devs devjunior
echo "devjunior:SenhaForte123!" | sudo chpasswd

# 3. Criação da estrutura de pastas da empresa
sudo mkdir -p /srv/projetos/backend

# 4. Configuração de propriedade: Dono root, Grupo devs
sudo chown -R root:devs /srv/projetos/backend

# 5. Aplicação da regra 750 (Dono tem tudo, Grupo lê/executa, Outros bloqueados)
sudo chmod 750 /srv/projetos/backend
ls -ld /srv/projetos/backend
```

Parte 2: Ciclo de Vida do Servidor Web e Trilha de Auditoria

```
# 1. Inicializar o Nginx e verificar se o daemon está operando
sudo systemctl start nginx
sudo systemctl status nginx --no-pager

# 2. Gerar eventos intencionais de tráfego (Sucesso 200 e Erro 404 Not Found)
curl -I http://localhost/
curl -I http://localhost/rota-teste-inexistente

# 3. Filtrar e exibir os eventos recém-registrados
sudo tail -n 3 /var/log/nginx/access.log
```

Parte 3: Coleta Rápida dos Sinais Vitais

```
uptime          # Avalia o Load Average imediato
free -h         # Verifica consumo real de RAM
df -hT /        # Checa ocupação da partição raiz
ss -tulpn | grep :80 # Confirma o socket do Nginx em escuta
```

Para viabilizar a execução dos exercícios sem exigir privilégios de virtualização em hardware (VT-x/AMD-V) nas máquinas do laboratório, utilizaremos contêineres Docker configurados com suporte integral ao systemd.

6.1. Especificação do Arquivo docker-compose.yml

Os alunos devem salvar a configuração abaixo na sua pasta de trabalho e inicializar o laboratório:

```
services:
  servidor-aula:
    image: ubuntu:24.04
    container_name: lab-linux-servidor
    privileged: true
    cgroup: host
    volumes:
      - /sys/fs/cgroup:/sys/fs/cgroup:rw
    ports:
      - "8080:80"
    command: ["/sbin/init"]
```

6.2. Comandos de Preparação e Acesso ao Ambiente

```
# 1. Subir o ambiente em segundo plano a partir do terminal do host
docker compose up -d

# 2. Entrar na sessão interativa como superusuário (root)
docker exec -it lab-linux-servidor bash

# 3. (Dentro do contêiner) Atualizar os repositórios e instalar os pacotes da aula
apt update && apt install -y sudo curl procps nano htop net-tools nginx
```

Validação do Mapeamento de Portas

Ao rodar o comando `systemctl start nginx` dentro do contêiner, o servidor web torna-se imediatamente acessível pelo navegador da máquina física acessando `http://localhost:8080`.

Cenário Corporativo: TechInovação S.A.

Você foi admitido como Administrador de Sistemas Júnior da *TechInovação S.A.* A diretoria de engenharia encarregou você de configurar as políticas de acesso do novo servidor de desenvolvimento, isolando diretórios confidenciais e ativando serviços web com monitoramento obrigatório.

Instruções para Entrega e Validação

- **Formato de Entrega:** Relatório técnico individual em formato PDF.
- **Evidências Necessárias:** Cada item exige a transcrição exata dos comandos digitados e a captura de tela (print) nítida do terminal exibindo o retorno dos comandos.
- **Prazo:** A atividade deve ser concluída e validada até o término da aula prática.

Questão 1: Gestão de Identidades e Grupos (20 Pontos)

1. Crie no sistema operacional um grupo local denominado `cienciadados`.
2. Crie dois novos usuários humanos: `analista_maria` e `analista_joao`. Ambos devem ter como grupo primário ou suplementar o grupo `cienciadados` e interpretador de comandos padrão `/bin/bash`.
3. Defina uma senha segura para ambos os usuários.
4. **Regra de Menor Privilégio:** Adicione **apenas** a usuária `analista_maria` ao grupo administrativo do sistema (grupo `sudo`). O usuário `analista_joao` não pode, sob hipótese alguma, possuir permissões de elevação via `sudo`.
5. *Evidência a anexar:* Comando `id analista_maria` e `id analista_joao` demonstrando os grupos aos quais pertencem.

Questão 2: Políticas de Segurança em Diretórios POSIX (25 Pontos)

1. Crie na raiz do sistema de arquivos o diretório corporativo `/dados_compartilhados`.
2. Configure o diretório para que pertença ao usuário `root` e ao grupo `cienciadados`.
3. Aplique a permissão octal restritiva `770` (`rw-rwx---`).
4. **Desafio de Isolamento:** Como usuário `root`, crie um arquivo de texto dentro desta pasta: `echo "Dados Confidenciais" > /dados_compartilhados/segredo.txt`.
5. Crie um usuário comum terceiro (ex: `estagiario`) sem vínculo com o grupo `cienciadados`. Simule o acesso deste usuário tentando listar a pasta: `su - estagiario -c "ls -la /dados_compartilhados"`.
6. *Evidência a anexar:* Saída do comando `ls -ld /dados_compartilhados` e o erro `Permission denied` obtido pelo usuário não autorizado.

Questão 3: Ativação de Serviço Web e Auditoria de Tráfego (35 Pontos)

1. Inicie o serviço do servidor web `nginx` e configure-o para iniciar automaticamente durante a inicialização do sistema através do comando `systemctl enable nginx`.
2. Comprove que o serviço está ativo executando `systemctl status nginx`.
3. A partir da máquina física do aluno (host), abra o navegador de internet e realize acessos ao endereço local do contêiner na porta 8080:
 - o Acesse a página inicial legítima: `http://localhost:8080/` (deve retornar a tela de boas-vindas do Nginx).
 - o Em seguida, force intencionalmente **3 acessos a arquivos inexistentes** para gerar eventos com código HTTP 404 (ex: `http://localhost:8080/folha_pagamento.pdf`, `http://localhost:8080/backup.sql` e `http://localhost:8080/admin_login`).
4. No terminal do servidor, filtre e extraia as últimas 10 linhas do arquivo de log `/var/log/nginx/access.log`, comprovando os códigos de retorno 200 e 404 associados aos acessos que você acabou de gerar.
5. *Evidência a anexar:* Print do status do Nginx e a listagem das requisições filtradas no log de acesso.

Questão 4: Diagnóstico de Recursos de Hardware (20 Pontos)

O SysAdmin precisa documentar a saúde do servidor recém-instalado. Execute os utilitários de sistema e monte no seu relatório uma tabela consolidando os seguintes indicadores:

1. **Consumo de Memória:** Quantidade total de memória RAM física do ambiente, memória utilizada e memória efetivamente disponível para novas aplicações (extraídas de `free -h`).
2. **Ocupação de Disco:** Sistema de arquivos, tamanho total e percentual ocupado na partição raiz (`df -hT /`).
3. **Gargalos de Processamento:** Média de carga da máquina nos últimos 1, 5 e 15 minutos (extraída de `uptime`).
4. **Mapeamento de Processos:** Liste os 3 processos que mais estão consumindo memória RAM no sistema no momento da avaliação, indicando seu PID, nome do comando e percentual de memória alocada.

8.1. Rubrica Oficial de Pontuação da Atividade

Item Avaliado	Critério Técnico de Aceite	Valor Máximo
Identities e Sudo (Q1)	Criação exata dos usuários, associação correta ao grupo cienciadados e privilégio sudo restrito exclusivamente a analista_maria.	20 Pontos
Permissões POSIX (Q2)	Propriedade root:cienciadados, máscara 770 e comprovação do bloqueio de leitura para contas não autorizadas.	25 Pontos
Serviço Web & Logs (Q3)	Nginx em execução e habilitado no boot; presença comprovada de requisições legítimas e requisições de erro 404 nos logs do servidor.	35 Pontos
Diagnóstico de Recursos (Q4)	Coleta e interpretação correta dos indicadores de memória RAM, partição raiz, carga de CPU e processos do sistema.	20 Pontos

8.2. Script de Validação Automática para o Aluno

O estudante pode auditar seu próprio contêiner antes de redigir o relatório final executando o validador interno:

```
# Baixar e executar o script de autocorreção dentro do contêiner
curl -s https://raw.githubusercontent.com/lab/validador.sh | bash
```

8.3. Coleta Automatizada das Notas pelo Professor

Para turmas numerosas, o instrutor pode executar a varredura em lote na máquina host, gerando o relatório consolidado em formato CSV:

```
for c in $(docker ps --format "{{.Names}}" | grep "lab-"); do
  docker cp validador_lab.sh $c:/tmp/validador.sh
  NOTA=$(docker exec $c bash /tmp/validador.sh | grep "PONTUAÇÃO FINAL" | grep -oE "[0-9]+")
  echo "$(date +%Y-%m-%d),Aluno_$c,$NOTA" >> consolidado_notas.csv
done
```